

BONUS 1 – Esplorazione di Nmap

Studente: Simone Santonico

Data: 19/06/2026

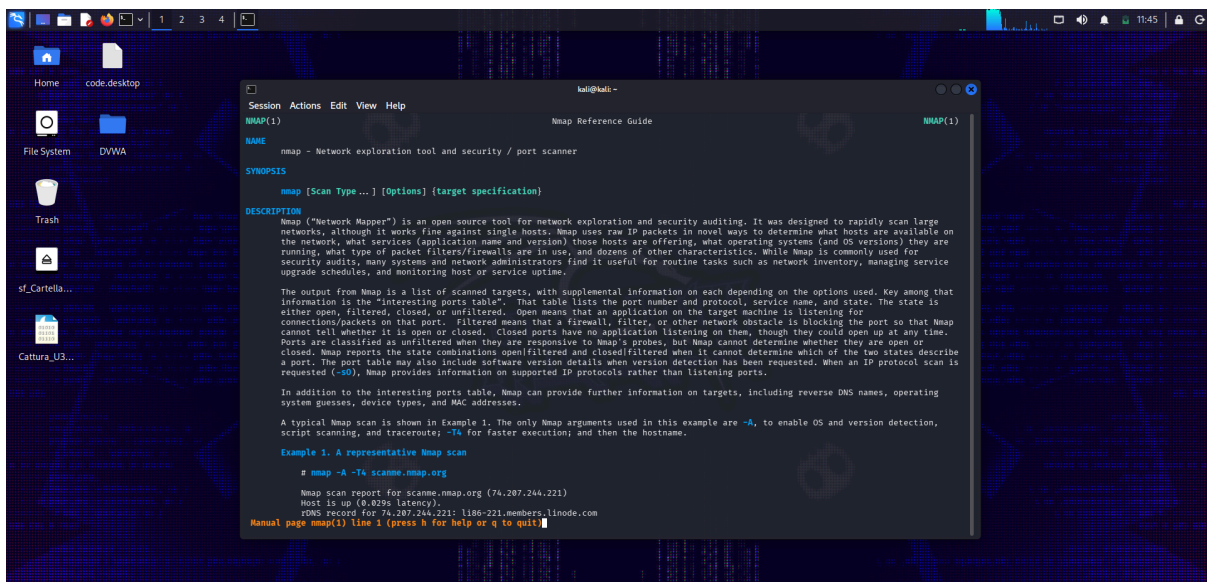
1. Cos'è Nmap?

Dal manuale ([man nmap](#)):

“Network exploration tool and security / port scanner”

Nmap è uno strumento open-source utilizzato per:

- Scansione porte
- Identificazione servizi e versioni
- Rilevamento sistema operativo
- Audit di sicurezza
- Inventario di rete



The screenshot shows a Kali Linux desktop with a dark theme. A terminal window titled 'kali@kali -' is open, displaying the 'Nmap Reference Guide'. The terminal content includes the following sections:

```
Session Actions Edit View Help
NMAP(1)                                Nmap Reference Guide

NAME
  nmap - Network exploration tool and security / port scanner

SYNOPSIS
  nmap [Scan Type ...] [Options] (target specification)

DESCRIPTION
  Nmap ("Network Mapper") is an open source tool for network exploration and security auditing. It was designed to rapidly scan large networks, although it works fine against single hosts. Nmap uses raw IP packets in novel ways to determine what hosts are available on the network, what services (application name and version) those hosts are offering, what operating systems (and OS versions) they are running, what type of packet filters/firewalls are in use, and dozens of other characteristics. While Nmap is commonly used for security audits, many systems and network administrators find it useful for routine tasks such as network inventory, managing service upgrade schedules, and monitoring host or service uptime.

  The output from Nmap is a list of scanned targets, with supplemental information on each depending on the options used. Key among that information is the "interesting ports table". That table lists the port number and protocol, service name, and state. The state is either open, filtered, closed, or unfiltered. Open means that an application on the target machine is listening for connections/packets on that port. Filtered means that a firewall, filter, or other network obstacle is blocking the port so that Nmap cannot tell whether it is open or closed. Closed ports have no application listening on them, though they could open up at any time. Ports are classified as unfiltered when they are responsive to Nmap's probes, but Nmap cannot determine whether they are open or closed. Nmap reports the state combinations open/filtered and closed/filtered when it cannot determine which of the two states describe a port. The port table may also include software version details when version detection has been requested. When an IP protocol scan is requested (-sO), Nmap provides information on supported IP protocols rather than listening ports.

  In addition to the interesting ports table, Nmap can provide further information on targets, including reverse DNS names, operating system guesses, device types, and MAC addresses.

  A typical Nmap scan is shown in Example 1. The only Nmap arguments used in this example are -A, to enable OS and version detection, script scanning, and traceroute; -T4 for faster execution; and then the hostname.

Example 1: A representative Nmap scan

# nmap -A -T4 scanme.nmap.org

Nmap scan report for scanme.nmap.org (74.207.244.221)
Host is up (0.029s latency).
DNS record for 74.207.244.221: 186-221.members.linode.com
Manual page nmap(1) line 1 (press h for help or q to quit)
```

2. Opzioni richieste

Dal manuale:

- **-A** → abilita OS detection, version detection, script scanning e traceroute
- **-T4** → aumenta la velocità della scansione con timing aggressivo

Queste opzioni sono state utilizzate in tutte le scansioni successive.

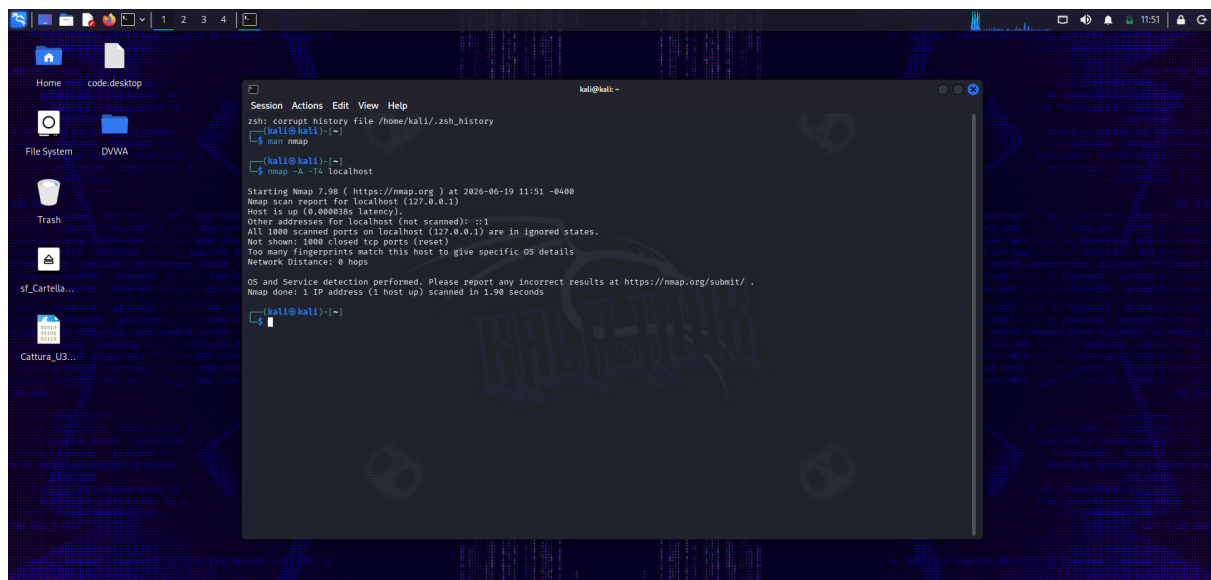
3. Scansione localhost

Comando eseguito:

```
nmap -A -T4 localhost
```

Risultati osservati:

- Host attivo
- Tutte le 1000 porte TCP risultano **chiuse**
- Nessun servizio in ascolto
- OS detection non precisa (normale quando non ci sono porte aperte)
- Network distance: 0 hops



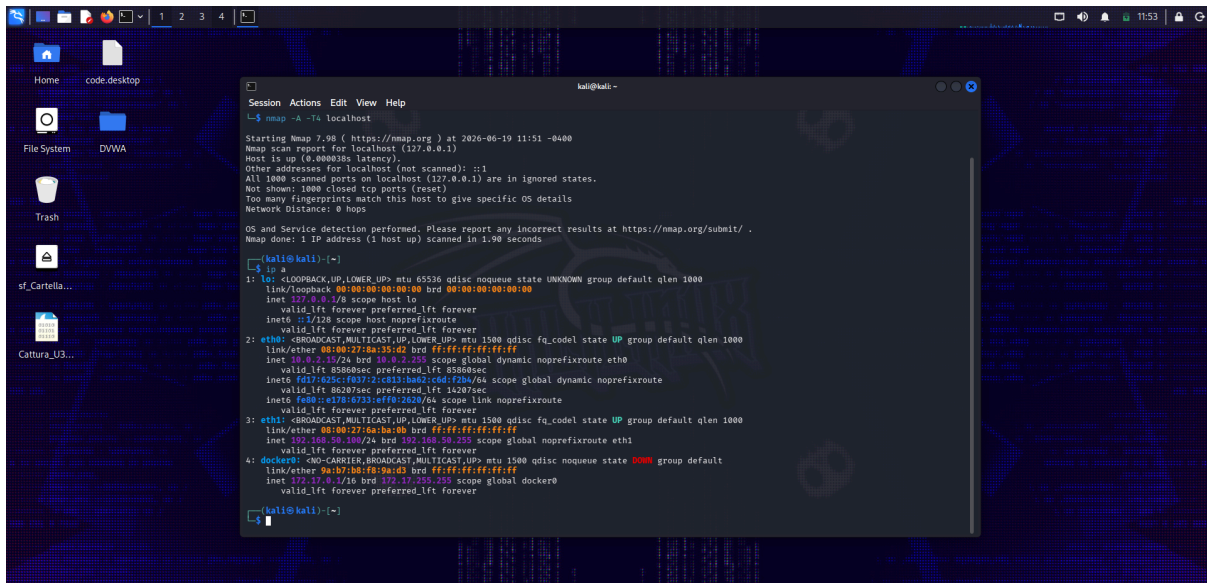
Conclusione:

Il sistema locale non espone servizi in ascolto, comportamento tipico di una installazione Kali Linux pulita.

4. Identificazione rete locale

Dal comando `ip a`:

- Interfaccia attiva: **eth1**
- IP: **192.168.56.10/24**
- Rete: **192.168.56.0/24**



```
kali@kali:~$ nmap -A -T4 localhost

Starting Nmap 7.98 ( https://nmap.org ) at 2020-06-19 11:51 -0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000038s latency).
Other addresses for localhost (not scanned): ::1
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Too many fingerprints match this host to give specific OS details
Network Distance: 0 hops

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.90 seconds

kali@kali:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0a:25:10 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 85608sec preferred_lft 85608sec
    inet6 fd17:02ec:f077:2:c81b:b4e2:cde:f2b/64 scope global dynamic noprefixroute
        valid_lft 86207sec preferred_lft 14207sec
    inet6 fe80::e78:6733:eff6:262b/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0a:b8:0a brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.10/24 brd 192.168.56.255 scope global noprefixroute eth1
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 9a:b7:b8:f8:0a:d3 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever

kali@kali:~$
```

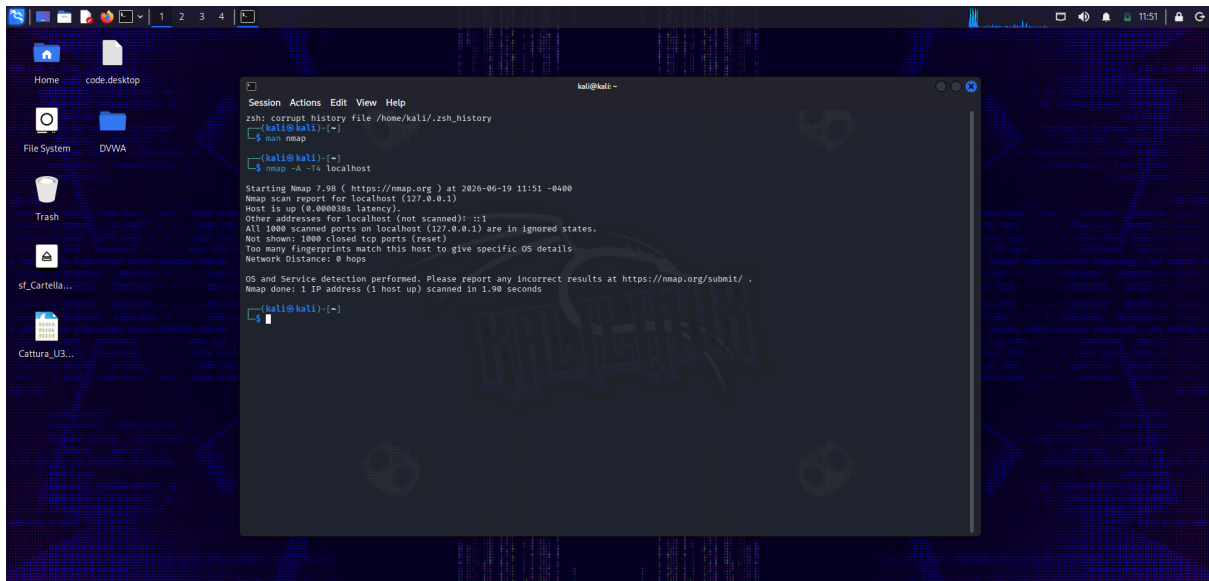
5. Scansione rete locale

Comando eseguito:

`nmap -A -T4 192.168.56.0/24`

Risultati osservati:

- Nmap rileva numerosi host attivi sulla rete VirtualBox Host-Only
- La scansione è lenta perché la rete contiene molti indirizzi attivi (64 host up)
- Alcuni host non rispondono a tutte le porte (warning: retransmission cap hit)



Conclusione:

La rete 192.168.56.0/24 contiene più macchine virtuali/host VirtualBox.

La scansione è stata avviata correttamente e ha rilevato numerosi host attivi.

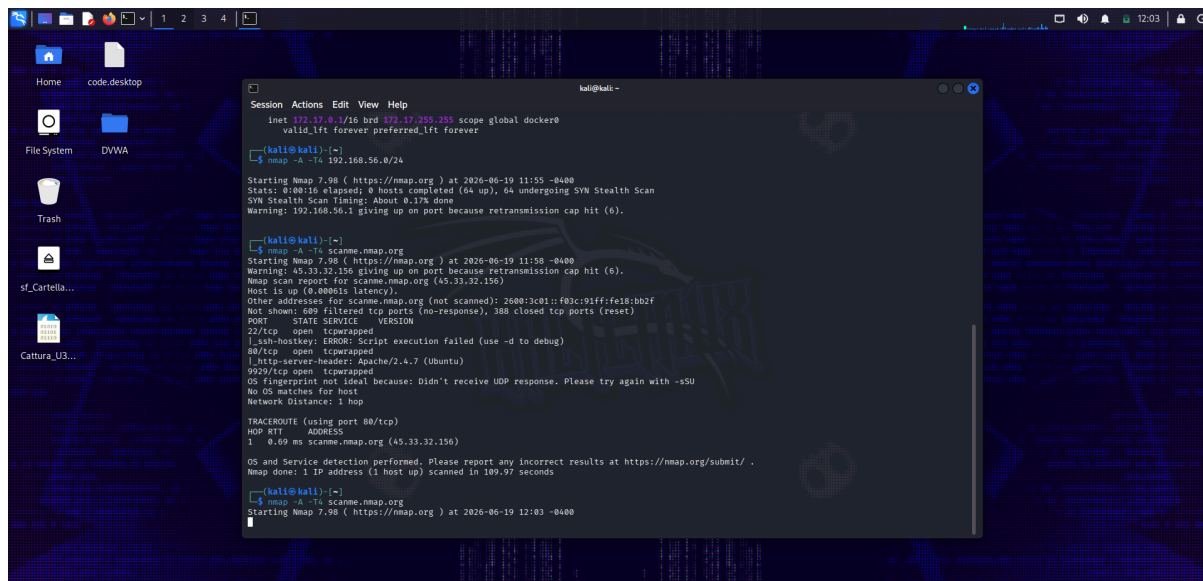
6. Scansione server remoto scanme.nmap.org

Comando eseguito:

```
nmap -A -T4 scanme.nmap.org
```

Risultati osservati:

- Host attivo: **45.33.32.156**
- Porte aperte rilevate:
 - **22/tcp** → ssh (tcpwrapped)
 - **80/tcp** → http (Apache/2.4.7)
 - **9999/tcp** → tcpwrapped
- Molte porte filtrate (609)
- OS detection non precisa
- Traceroute: 1 hop (connessione diretta)



Conclusion:

Il server pubblico di test di Nmap espone poche porte controllate e molte porte filtrate, comportamento tipico di un host sicuro e monitorato.

7. Conclusione generale del Bonus 1

Il Bonus 1 ha permesso di:

- Comprendere il funzionamento di Nmap tramite il manuale
- Eseguire scansioni locali e remote
- Identificare porte, servizi e host attivi
- Analizzare una rete VirtualBox
- Osservare un server remoto reale (scanme.nmap.org)

Le scansioni sono state eseguite correttamente e i risultati sono coerenti con un ambiente di laboratorio.

BONUS 2 – Analisi Attacco SQL Injection (PCAP)

1. Introduzione

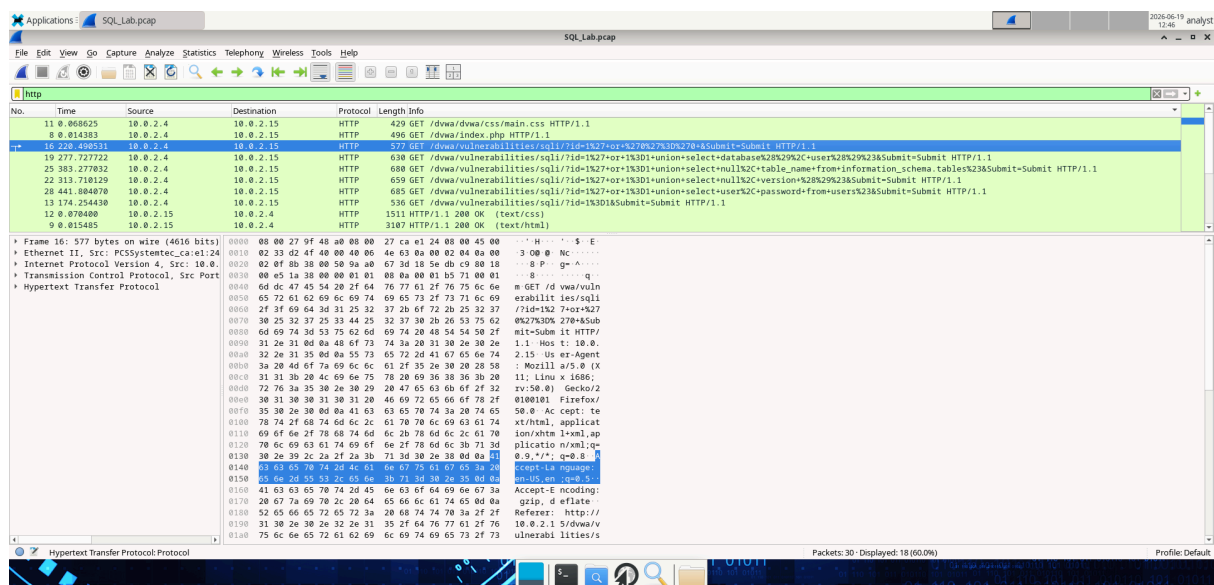
In questo bonus è stato analizzato il file **SQL_Lab.pcap**, contenente il traffico generato da un attacco **SQL Injection** contro l'applicazione vulnerabile **DVWA (Damn Vulnerable Web Application)**.

L'obiettivo è identificare le fasi dell'attacco osservando le richieste HTTP e le risposte del server.

2. Accesso alla pagina vulnerabile

L'attaccante accede alla pagina DVWA vulnerabile alla SQL Injection:

GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit

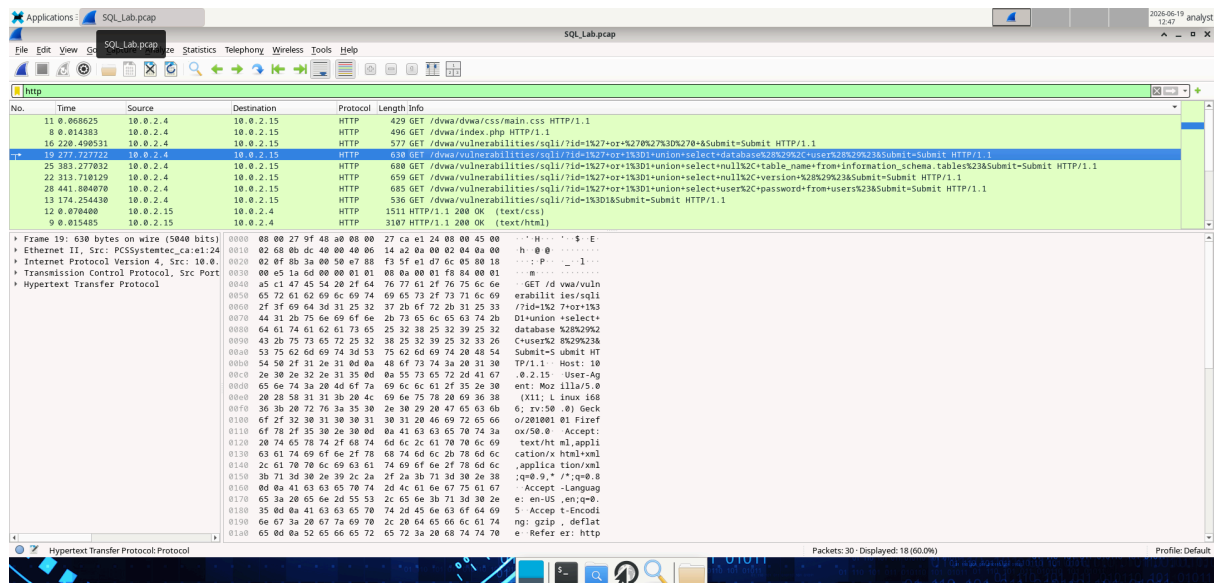


3. Fase di exploit – SQL Injection

L'attaccante modifica il parametro `id` inserendo un payload SQL malevolo:

`?id=1' or '0'='0`

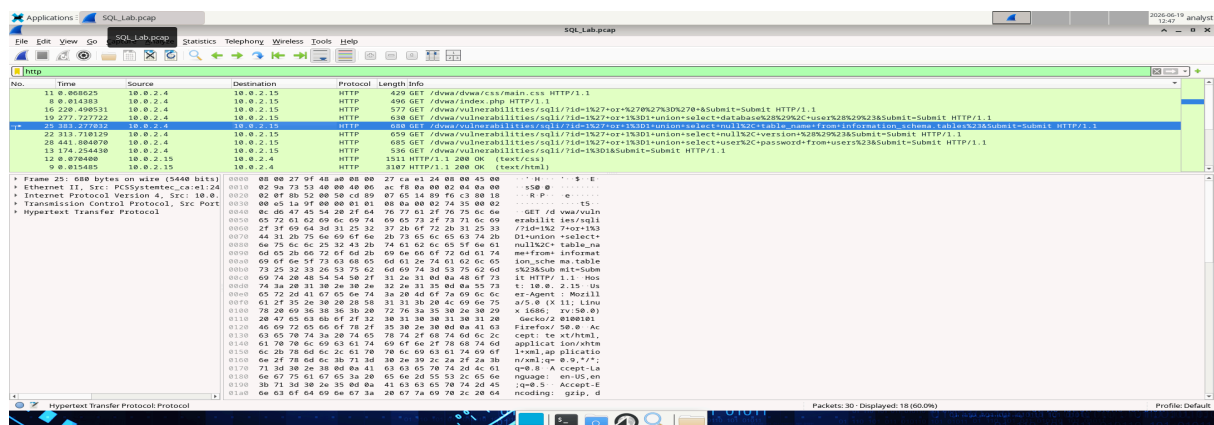
Questa iniezione altera la query SQL rendendo la condizione sempre vera e bypassando i controlli applicativi.



4. Enumerazione del database

Dopo aver verificato la vulnerabilità, l'attaccante utilizza una UNION SELECT per ottenere informazioni sul database:

`?id=1' or 1=1 union select database()#`



Questa richiesta permette di identificare il nome del database attualmente in uso.

5. Estrazione delle tabelle

L'attaccante procede con una seconda UNION SELECT per estrarre i nomi delle tabelle dal database:

```
?id=1' or 1=1 union select null, table_name from information_schema.tables#
```

Questa tecnica consente di ottenere la struttura del database, utile per attacchi successivi.

6. Conclusione

L'analisi del file PCAP mostra chiaramente tutte le fasi di un attacco SQL Injection:

- accesso alla pagina vulnerabile
- iniezione SQL
- enumerazione del database
- estrazione delle tabelle

L'attacco ha avuto successo grazie alla mancanza di sanitizzazione del parametro `id`, permettendo all'attaccante di manipolare la query SQL e ottenere informazioni sensibili dal database.